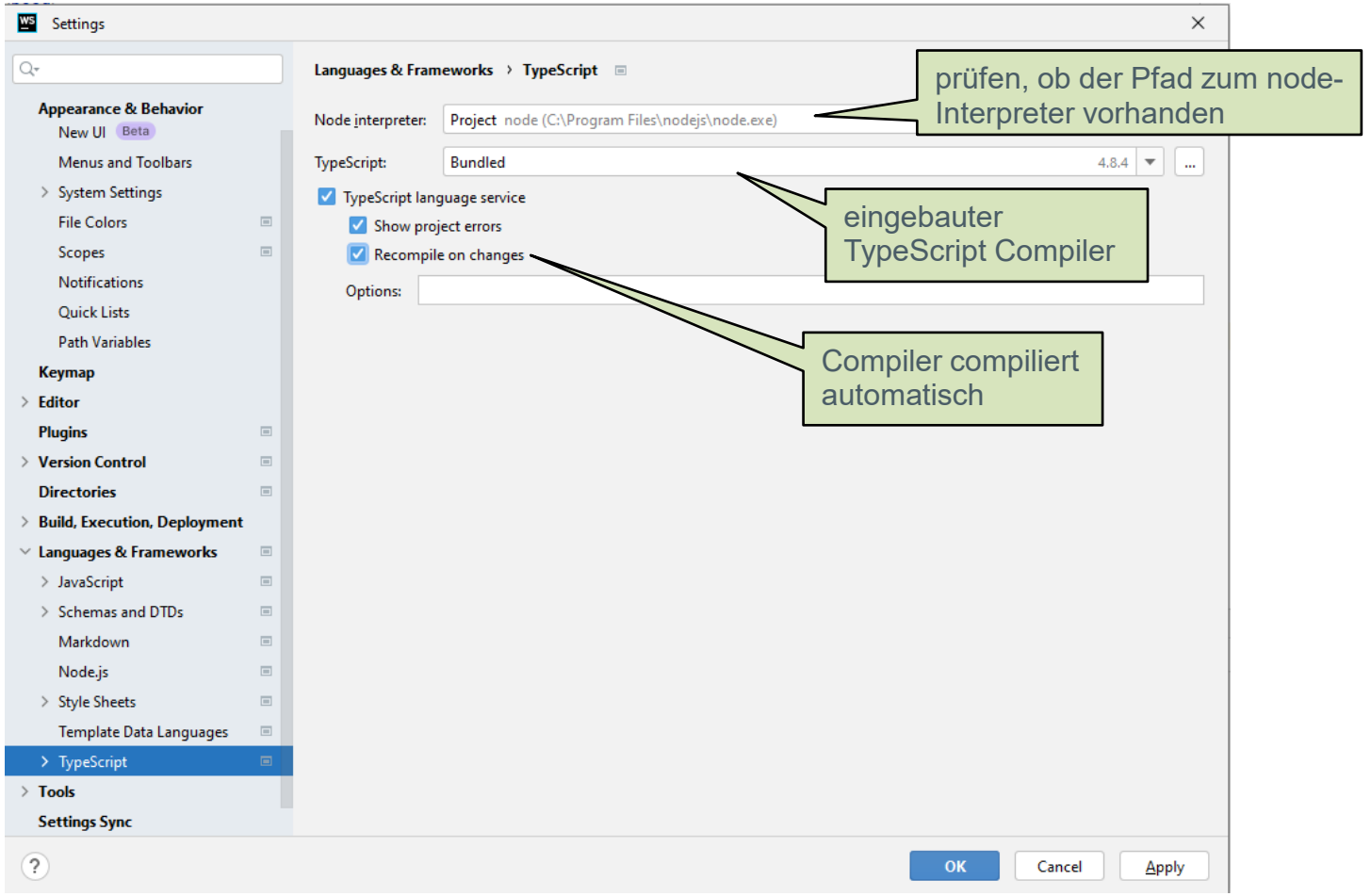
1.	Organisatorisches	4
2.	Einleitung	21
3.	HTML	46
4.	CSS	70
5.	Frameworks	89
6.	HTTP	106
7.	Internet	126
8.	**TypeScript**	**178**
9.	HTML+CSS+Typescript	224
10.	Web Services	235
11.	REST-Services	258

Konfiguration

In WebStorms:
-> File
-> Settings
-> Languages & Frameworks
-> TypeScript



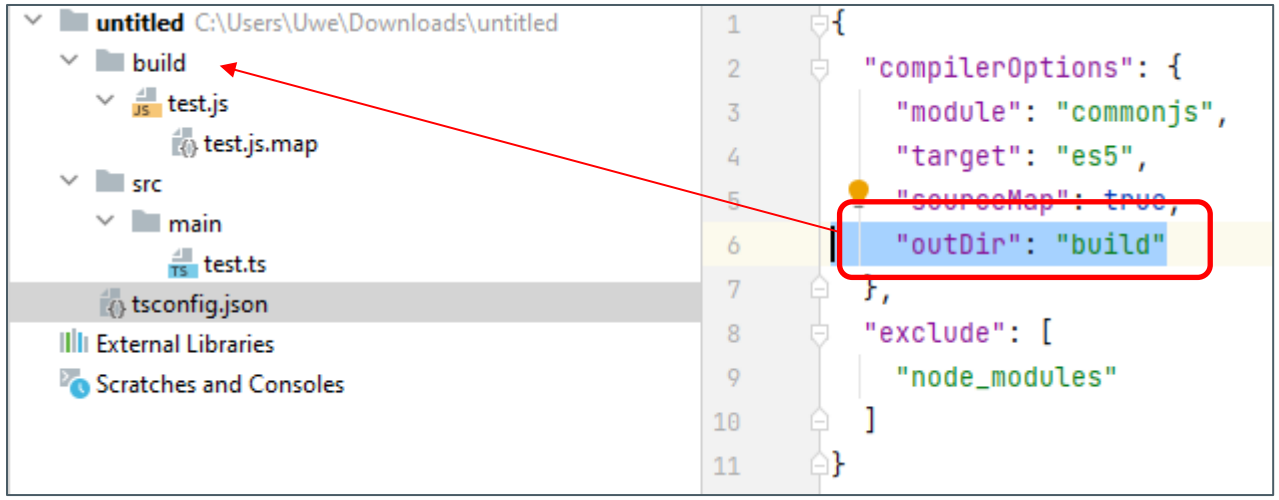
Konfiguration (3)

Editieren Sie die `tsconfig.json`-Datei und ergänzen Sie unter „compilerOptions“:

"outDir": "build“

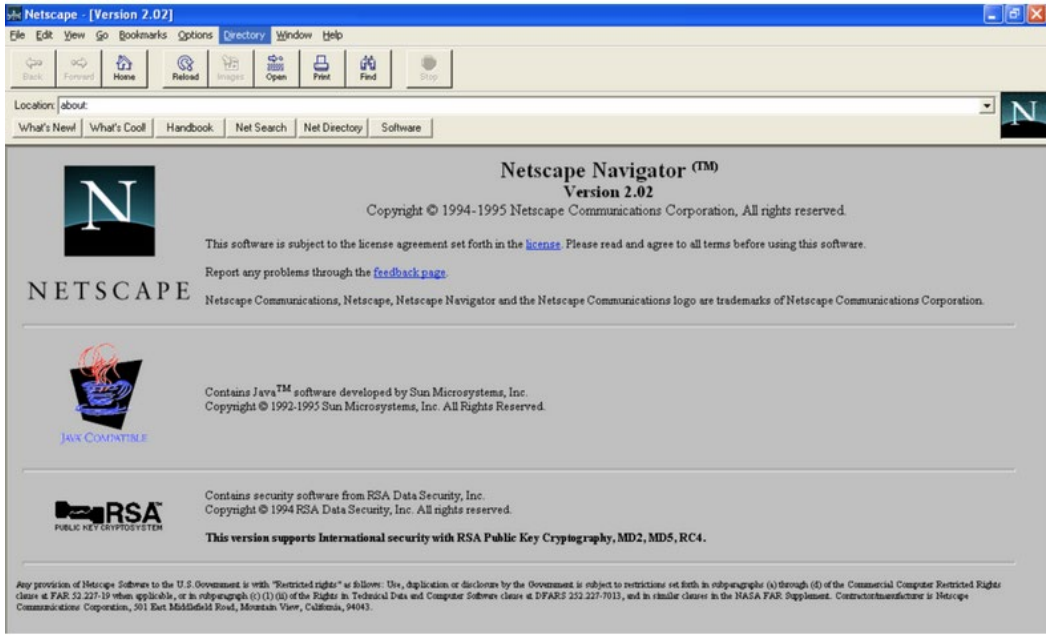
(Denken Sie an das Komma am Ende der Zeile darüber!)

Jetzt sollte beim Kompilieren ein Ordner mit dem Namen build angelegt wird, in dem die JavaScript-Dateien abgelegt werden.



JavaScript ≠ Java

JavaScript



- Im April 1995 entwickelte Brendan Eich eine Skriptsprache im Netscape-Browser, die er Mocha nannte und die im September gemeinsam mit Sun Microsystems aus Marketing-Gründen in JavaScript umbenannt wurde.
- In 1997 übergab Netscape die Standardisierung an die ECMA.
- JavaScript ist die Implementierung des ECMAScript Standards.
- TypeScript wurde von Microsoft entwickelt und erweitert JavaScript im Wesentlichen um Typen.

JavaScript in HTML einbinden

1. Referenz auf Javascript im Header – die sauberste Lösung insb. für größere Projekte

```
<head>
  <script src="folder/file.js"></script>
</head>
```

2. Javascript direkt im Header – die schnelle Lösung für kleinere Projekte

```
<head>
  <script language="javascript" type="text/javascript">
  ... // some javascript code here
  </script>
</head>
```

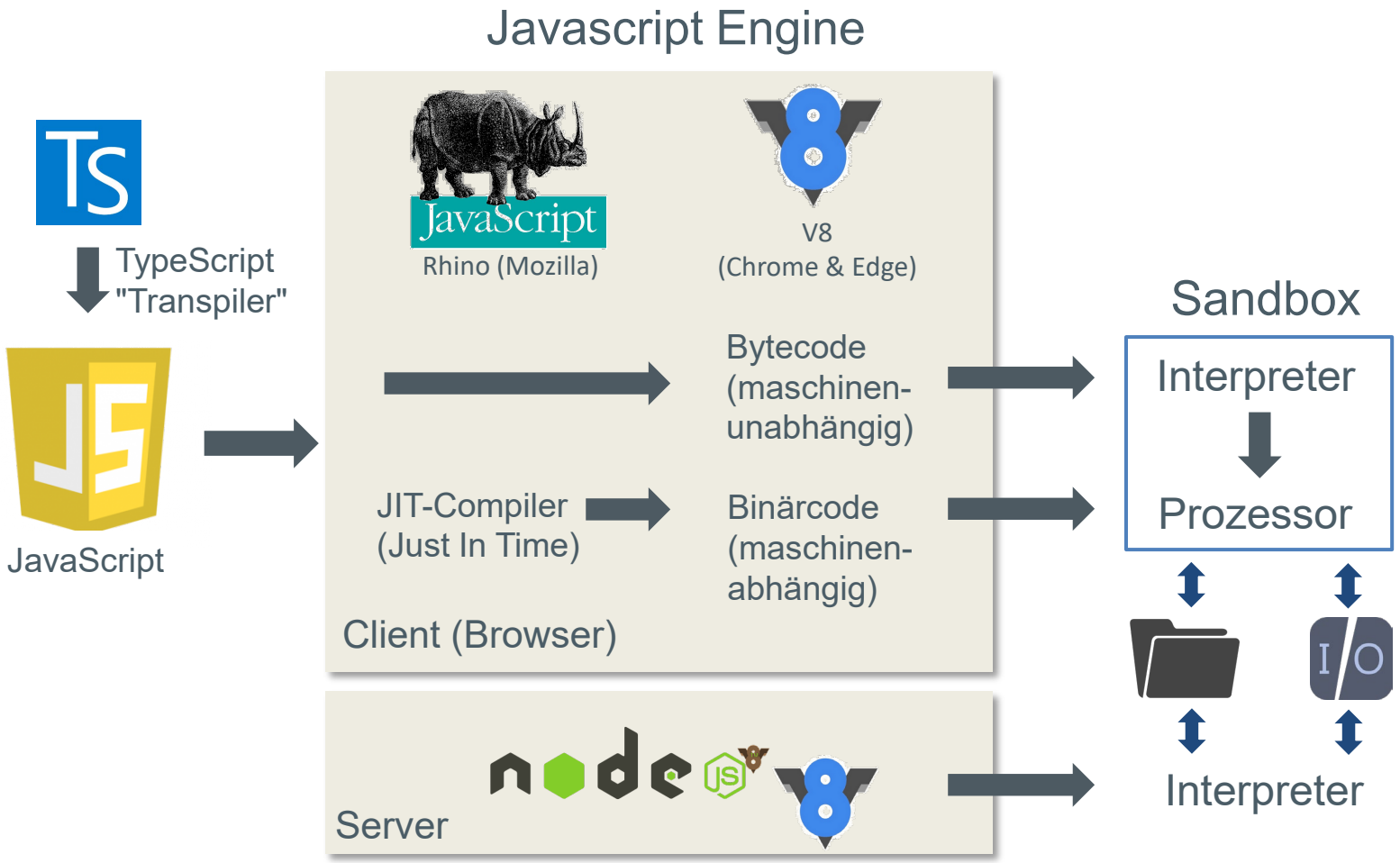
   - Javascript-Code liegt in gleicher Datei – wird damit immer neu geladen

3. Javascript im HTML-Element – die kleine Lösung

```
<tag JS-Event="JavaScript-Programm;"> ... </tag>
Bsp: <body onclick="alert('nervige Alert-Box')"> .... </body>
```

   - nur für „vereinzelte“ JavaScript-Anweisungen
   - ... übersieht man gerne beim "debuggen"

Ausführung von JavaScript



Variablendeklarationen

- Syntax:

Variablen - Bezeichner *Typ - Bezeichner*

```
let <Name>: <Typ> [= <Wert ];
```

- Beispiele:

```
let score: number;
let myName: string = ‘Otto’;
```

- Mehrere Variablen deklarieren:

```
let firstName: string, lastName: string;
```

Zuweisungen

- Syntax
  ```
  <Name> = <Ausdruck>;
  ```

- Beispiele
  ```
  let score: number;
  score = 50;
  score = score * 2 ;
  ```

- Zuweisung eines Wertes, welcher nicht zum Typ der Variable passt, ist ein Programmierfehler.
  ```
  let firstName: string;
  firstName = 50;
  ```

  Was passiert?

  Fehlermeldung: "TS2322: Type 'number' is not assignable to type 'string'."

Unveränderliche Variablen (Konstanten)

- Deklaration als Konstante (keine nachträgliche Zuweisung möglich)

```
const groesse: number = 8;
groesse = 100;
```

  Fehlermeldung: "TS2588: Cannot assign to groesse' because it is a constant.„

- Hinweis: Das Schlüsselwort `var` **nicht** benutzen.
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/let#scoping_rules

Datentypen

- In JavaScript werden die Typen einer Variablen durch die erste Zuweisung dynamisch (zur Laufzeit) festgelegt und muss vorher nicht bekannt sein. Damit ist man während der Programmierung nicht gezwungen, sich über Strukturen, Wertebereiche etc. Gedanken zu machen, was zu unstrukturierten Programmen und zu (teuren) Fehlern zur Laufzeit führen kann.

- JavaScript-Beispiel:

```
let x = 1;
x = x + “2“ + [3,4,5];
```

- TypeScript verwendet Typen, die während der Programmierung bedacht, zugewiesen und vom Compiler überprüft werden.
  Bei dem obigen Beispiel haut Ihnen der TypeScript-Compiler auf die Finger!

Number

- Zur Darstellung ganzer Zahlen (mit oder ohne Vorzeichen) und von Fließkommazahlen
- Belegen 64 Bit im Speicher
- Number-Literale können dezimal, hexadezimal, binär oder mit Mantisse und Exponent geschrieben werden:
  `-42, 5.99, 0xFF00FF, 0b01000001, 1.5104e11`

String

- Zur Darstellung von beliebig langen Zeichenketten
- Wertebereich: Zeichenketten aus Unicode-Zeichen

```
let header: string = "Willkommen!";
```

- Initialisierung per Literal
  - Einfache Anführungszeichen erlauben die Nutzung von Doppelten Anführungszeichen und umgekehrt.

```
let complexHeader: string = 'Hallo “Otto".’;
```

  - Template-Syntax ${…} ermöglicht das Verschachteln von Strings. Achtung: Backticks `…` benutzen!

```
let userName: string = “Heinz!";
let welcomeMessage: string = `Hallo, ${userName}.`;
```

  - Zeilenumbruch (\n)

```
let userName: string = “Willy";
let badgeNumber: number = 42;
let profile: string
     = `Name: ${userName}\nBadge Number: ${badgeNumber.toLocaleString("de")}`;
```

Operatoren auf Strings

- Konkatenation (Aneinanderhängen von Zeichenketten per +)
```
let fullName: string = firstName + " " + lastName;
```

- Zeichenanzahl geben lassen (.length)
```
let characterCount: number = fullName.length;
```

- Konvertierung in Großbuchstaben (.toUpperCase).

- Entfernung von Leerzeichen (Entfernt Leerzeichen am Anfang und am Ende eines Strings)
```
let userInput: string = "  mymail@gmail.com ";
let cleanInput: string = userInput.trim(); // mymail@gmail.com
```

- Konvertierung string → number (Ganzzahl)
```
let userInput: string = “35390";
let numberValue: number = parseInt(userInput); // 35390
```

Boolean

- Zur Darstellung von Wahrheitswerten true oder false
- Initialisierung per Literal oder Ausdruck

```
let isSubscribed: boolean = true;
let isAdmin: boolean = userID !== 0;
```

- Operatoren
  - "Und" (true, wenn beide Operanden true sind. Ansonsten false.)

```
let didAcceptTermsAndConditions: boolean = true;
let didAcceptPrivacyNotice: boolean = false;
let canContinue: boolean = didAcceptTermsAndConditions && didAcceptPrivacyNotice; // false
```

  - "Oder" (true, wenn einer der Operanden oder beide Operanden true sind. Ansonsten false.)

```
let isBlocked: boolean = false; let isMuted: boolean = true;
let shouldHidePost: boolean = isBlocked || isMuted; // true
```

  - "Nicht" (Invertiert den Wert.)

```
let shouldDisplayErrorMessage: boolean = !canContinue; // true
```

  - *Zuweisung*

```
let isEqual: boolean = index >= 0 || isMuted;
```

Übung

a) Erstellen Sie zwei Variablen und weisen Sie ihnen Ihren Vornamen und Nachnamen zu.

Geben Sie die Summe der Längen des Vornamens und des Nachnamens aus.

```
let vorname : String= "Uwe"
let nachname : String = "Meyer"
console.log("Länge Ihres Namens:" + (vorname.length+nachname.length));
```

b) Mit eckigen Klammern kann man auf die einzelnen Zeichen eines Strings zugreifen.
Beispiel: vorname[0] gibt den ersten (!) Buchstaben des Vornamens zurück.

Ermitteln Sie den ersten Buchstaben des Vornamens und den letzten des Nachnamens und vertauschen Sie diese.

Beispiel: Uwe Meyer -> rwe MeyeU

```
let ersterBuchstabe = vorname[0];
let letzterBuchstabe = nachname [nachname.length-1];
vorname = letzterBuchstabe + vorname.slice(1);
nachname = nachname.slice(0,nachname.length-1) + ersterBuchstabe;
console.log("Neuer Name: " + vorname + nachname);
```

Array

- Arrays sind eine sortierte Liste von Werten
  - Die Werte eines Arrays müssen vom gleichen Typ, dem *Basistyp* sein
  - Die Anzahl der Elemente ist beliebig und kann im Verlauf des Programms verändert werden.



Deklaration

- Ein Array kann auf zwei äquivalente Weisen deklariert werden

```
let names: Array<string>;
let names: string[];
```

- Ein Array sollte mindestens mit einer leeren Liste initialisiert werden.

```
let names: string[] = [];
```

- Initialisierung mit Werten

```
let names: string[] = [“Otto", “Heinz", “Paul"];
```

Übung

a) Deklarieren Sie ein zweidimensionales 3 x 3 Feld, das Zahlen enthalten soll!
b) Initialisieren Sie es mit den Zahlen von 1 bis 9!

```
let feld : Array<Array<number>>;          // oder number[][]

let feld : Array<Array<number>> = [[1,2,3], [4,5,6], [7,8,9]];
```

Operationen auf arrays

```
var names = ["Otto", "Heinz", "Paul"];

names.push("Karl");                                  // [Otto, Heinz, Paul, Karl]
console.log(names[1]);                               // Heinz

names[2] = "Willy";
console.log("Anzahl: " + names.length);              // 4

names.splice(2, 1);                                  // löscht ab Index 2 ein Element
console.log(names);                                  // Otto, Heinz, Karl
```

Enum (Aufzählungstypen)

- zur Darstellung beliebiger Werte eines diskreten Wertebereichs
- Deklaration:
```
enum Color { red, green, blue }
```
- Nutzung:
```
let myFavColor: Color = Color.green;
let yourFavColor: Color = Color.blue;
let weAreCompatible: boolean = myFavColor === yourFavColor; // false
```
- Wertebereich wie in der Deklaration des Typs angegeben
- Operationen: Keine
  - d.h. man kann Werte eines Aufzählungstyps nur zuweisen und vergleichen
- Interne Kodierung
  - Die Werte eines Enum-Types werden intern als Ganzzahl abgelegt. (Kann konfiguriert werden)
```
console.log(Color.green); // 1
```

Tupel

- Tupel sind eine geordnete Zusammenfassung von Daten
  - Die Elemente eines Tupels können von unterschiedlichem Typen sein
  - Die Anzahl der Elemente ist durch die Angabe des Basistypen festgelegt.
- Operation: Auswahl
  - Die Auswahl eines Elementes erfolgt über einen ganzzahligen Index über den (Auswahl-)Operator „ [ ] “
- Deklaration und Initialisierung:

```
let person1: [string, string, number] = [“Otto", “Schmidt", 3442];
let person2: [string, string, number] = [“Heinz", “Müller", -5];
let person3: [string, string, number] = [“Willy", “Schultz", 0];
```

- Elemente eines Tupels auswählen:

```
let fullNameOfPerson1: string = person1[0] + " " + person1[1];
// “Otto Schmidt"
```

- Element eines Tupels verändern:

```
person3[0] = “Karl"; // [“Karl", “Schultz", 0]
```

Typaliase

- Oft verwendet man einen zusammengesetzten Typen an mehreren Stellen, dann kann man mit einem Typalias für diesen Typen einen Namen vergeben.
- Beispiele

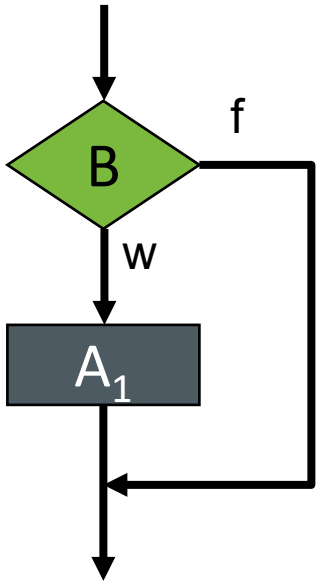
```
type paar = [string, number];
type id = number | string;
type myNumber = number;
type matrix = Array<Array<number>>;
```

- Typaliase dienen der Abstraktion und erhöhen die Lesbarkeit des Programms!

Auswahl (if)

Eine Aktion wird in Abhängigkeit einer booleschen Bedingung ausgeführt oder nicht („einarmiges if“).

Flussdiagramm



Pseudocode

```
if    (Bedingung)
{
    A1;
}
```

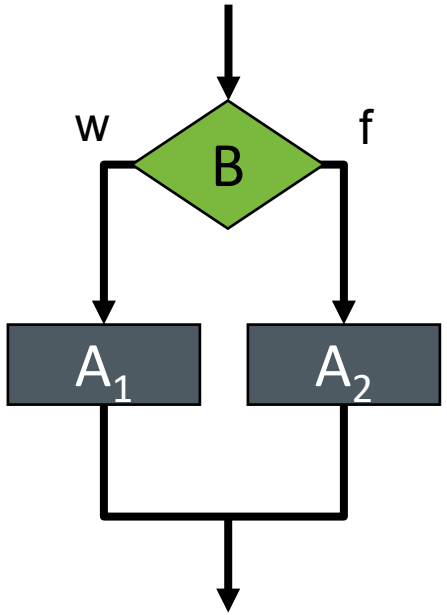
Beispiel

```
<script>
  let preis = parseInt(prompt("Preis? ", 0));
  let versandkosten = 0;
  if (preis < 100) {
    versandkosten = preis*0.05;
  }
  alert("Versandkosten: " + versandkosten + "€");
</script>
```

Auswahl (if)

In Abhängigkeit einer booleschen Bedingung wird eine Aktion oder eine andere ausgeführt („zweiarmiges if“).

Flussdiagramm



Pseudocode

```
if    (Bedingung)
{
   A1;
}
Else {
   A2;
}
```

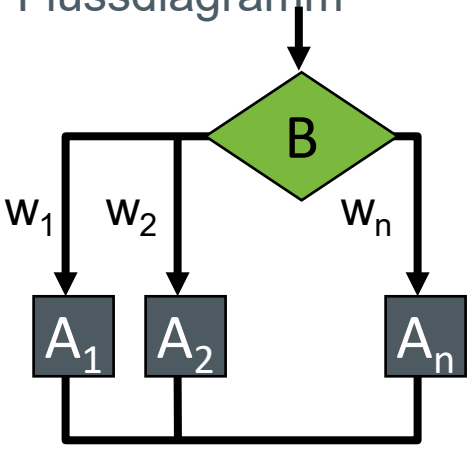
Beispiel

```
<script>
  let preis = parseInt(prompt("Preis? ", 0));
  let versandkosten = 0;
  if (preis < 100) {
    versandkosten = preis*0.05;
  }
  else {
    versandkosten = preis * 0.02;
  }
  alert("Versandkosten: " + versandkosten + "€");
</script>
```

Mehrfachauswahl (switch)

In Abhängigkeit der Auswertung eines Ausdrucks (mit mehreren möglichen Werten w_1, w_2, ..., w_n) wird eine Aktion aus einer Menge möglicher Aktionen ausgewählt und ausgeführt. Der Wertebereich muss dabei diskret sein.

Flussdiagramm



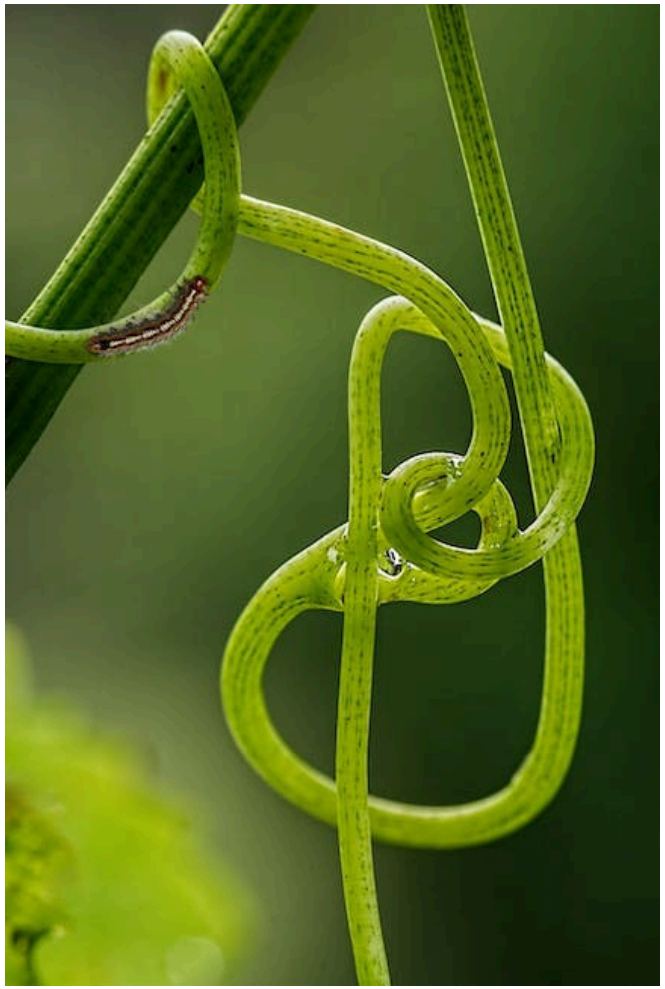
Pseudocode

```
switch (Ausdruck){
  case w1:
    A1;
    break;
  case w2:
    A2;
    break;
  ...
  case wn:
    An;
    break;
  default:
    Adefault;
}
```

Beispiel

```
let a: string = "blau";
switch (a) {
  case "grün":
    alert("a ist grün.");
    break;
  case "rot":
    alert("a ist rot.");
    break;
 case "gelb":
    alert("a ist gelb.");
    break;
  default:
    alert("Farbe unbekannt.");
}
```

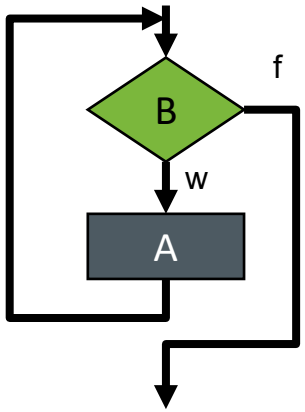
Schleifen



Schleifen erlauben die mehrfache Ausführung von Aktionen:

- Kopf-gesteuerte Schleifen (while-loops)
- Fuß-gesteuerte Schleifen (do-while-loops)
- Zähler-gesteuerte Schleifen (for-loops)

while-Schleifen

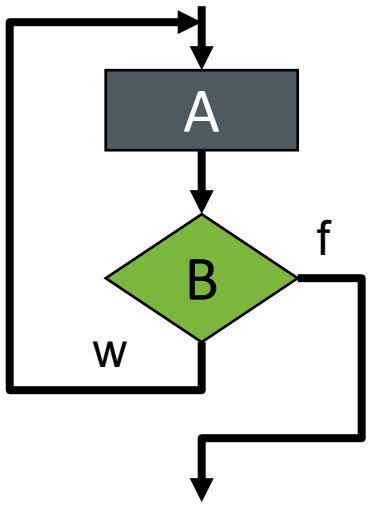


- Die Aktion A wird ausgeführt so lange ausgeführt, wie die Bedingung B wahr ist.
- Wenn B zu false ausgewertet wird, wird die nächste Anweisung nach der Schleife ausgeführt.
- Beispiel:

```
let n: number = 10;
let summe: number = 0;
let i: number = 1;

while (i <= n) {
  summe = summe + i;
  i = i + 1
}
```

do-while-Schleifen



- Die Aktion A wird ausgeführt. Wenn die Bedingung B wahr ist, wird A nochmals ausgeführt, bis B falsch ist.
- Wenn B zu false ausgewertet wird, wird die nächste Anweisung nach der Schleife ausgeführt.
- Beispiel:

```
let n: number = 10;
let summe: number = 0;
let i: number = 1;

 do {
   summe = summe + i;
   i = i + 1;
} while (i <= n);
```

Ergebnis?

for-Schleifen

- Sind Anfangs- und Endwert der Schleife vorgegeben, ist oft eine for-Schleife einfacher:
- Syntax:

```
for ([<Zuweisung>]; [<Bedingung>]; [<Zuweisung>]) {
        Aktion;
}
```

- Beispiel:

```
for (i=1; i<=n; i++) { summe += i; }
```

- Beachten Sie, dass alle drei Bestandteile optional sind!
- Beispiel:

```
for (;;) { … }
```

for-of

Oft möchte man in einer Schleife auf die Werte einer Liste zugreifen.

```
for (wert of liste) {
    Verwendung von Wert in Aktion;
}
```

Beispiel:

```
const prims = [2,3,5,7,11];
for (let p of prims) console.log(p);
```